多 Agent 协作：从个体到群体智能

- 来源: https://class.imooc.com/lesson/5292#mid=105155
- 阶段: 阶段六: 【智能体】从单体到多体
- 周: 第12周 Multi-Agent：多智能体协作
- 模块: 多Agent协作：从个体到群体智能
- 课节: 图文 1-1 多 Agent 协作：从个体到群体智能

1-1 多 Agent 协作：从个体到群体智能

多 Agent 协作：从个体到群体智能

> [!TIP] 阅读指南 在前面几节中，我们深入剖析了单个Agent的内部机制——从感知、思考到行动，从记忆系统到推理模式，再到各种实现范式。你现在已经能够构建一个功能完整的单体Agent。 但当你真正面对复杂的业务场景时，你会发现：一个Agent再强大，也有其能力边界。 举个典型的场景：你要开发一个内容营销系统。它需要研究市场趋势、撰写文章、设计配图、审核内容、制定发布计划。理论上，你可以训练一个超级Agent来完成所有这些任务。但实际上，这样做既低效又容易出错——就像让一个人既当设计师、又当编辑、还当市场分析师一样。 更好的方式是什么？将这个复杂任务拆解给多个专业Agent——市场研究Agent、内容创作Agent、设计Agent、审核Agent、策略规划Agent。每个Agent专注于自己擅长的领域，然后通过协作完成整体目标。 这就是多Agent系统的核心理念：从"个体智能"走向"群体智能"。 但这也带来了新的挑战：如何让多个Agent高效协作？如何避免它们各自为政？如何确保信息在它们之间准确传递？ 本节我们将探讨这些问题。

为什么需要多Agent?

单Agent的三大局限

先从单Agent的问题说起。

局限一：能力边界

就像人有专业领域一样，单个Agent也有其擅长和不擅长的任务。用一个通用Agent处理多种类型的任务时，它在每个任务上的表现都可能不如专门训练的Agent。

一个擅长写代码的Agent，让它写营销文案可能不够专业；一个擅长数据分析的Agent，让它做创意设计可能力不从心。

你当然可以投喂更多训练数据，让它"全能"。但这会导致模型规模膨胀、成本急剧上升，训练难度加大、各能力之间可能相互干扰，维护成本高、难以针对特定能力优化。

局限二：上下文窗口限制

即使是最新的大模型，上下文窗口也是有限的。当任务涉及大量背景信息、多个步骤的中间结果时，单Agent的上下文很容易被撑爆。

比如处理这样的任务：分析10篇竞品报告（每篇3000字），根据分析结果设计产品策略，撰写产品需求文档，生成开发任务清单。如果用单Agent，它需要在上下文中同时装载所有报告、分析结果、策略思考、文档草稿……很快就会触及Token上限。

局限三：并行处理能力

单Agent是串行工作的——完成步骤A，才能开始步骤B。但很多任务天然就是可以并行的。

比如分析一批用户评论：分析第1-100条评论的情感倾向、第101-200条、第201-300条、第301-400条——这些子任务之间相互独立，完全可以同时进行。用单Agent串行处理，效率太低。

多Agent的四大优势

多Agent系统正是为了突破这些局限而生。

优势一：专业化分工

每个Agent可以专注于自己最擅长的领域，就像一个专业团队：市场研究Agent精通数据分析、趋势预测；内容创作Agent擅长文案撰写、故事讲述；技术审核Agent专注代码审查、技术把关；项目管理Agent负责任务分配、进度跟踪。

这样做的好处是：每个Agent可以用更小、更高效的模型，针对特定领域优化效果更好，且易于维护和升级。

优势二：知识边界清晰

每个Agent维护自己的知识库和上下文，不会相互干扰。当某个领域的知识更新时，只需要升级对应的专业Agent，而不是重新训练整个大模型。

优势三：并行处理

多个Agent可以同时工作，大幅提升效率。就像前面举例的技术分享案例，PPT、演讲稿、案例、Q&A可以由四个Agent同时准备，然后汇总。

优势四：容错与备份

当某个Agent出现问题时，不会导致整个系统瘫痪。其他Agent可以继续工作，甚至可以由备用Agent接管任务。这在生产环境中非常重要。

多Agent协作的四种模式

理解了多Agent的价值，下一个问题是：如何组织它们协作?

业界总结出了四种主流的协作模式，每种适用于不同的场景。

层级协作（Hierarchical）

这是最常见的模式，类似于公司的组织架构。

核心思想：有一个Manager Agent负责任务分配和结果整合，多个Worker Agent负责执行具体任务。

工作流程：用户将任务提交给Manager Agent；Manager分析任务，拆解成子任务；Manager将子任务分配给不同的Worker Agent；Worker Agent各自完成任务，返回结果；Manager整合所有结果，生成最终输出。

适用场景：任务可以明确拆解为子任务、子任务之间相对独立、需要统一协调和质量把控。

劣势：Manager成为瓶颈，需要很强的任务分解能力；Worker之间无法直接交流，灵活性受限。

平等协作（Peer-to-Peer）

这种模式下，所有Agent地位平等，通过协商来完成任务。

核心思想：没有中心控制者，Agent之间自由对话、互相帮助，共同推动任务完成。

工作流程：所有Agent接收到同一个任务目标；Agent之间轮流发言，提出观点或方案；其他Agent对方案进行评论、补充或质疑；经过多轮讨论，逐步形成共识；达成一致后，输出最终方案。

适用场景：需要集思广益、头脑风暴，没有明确的任务拆解方式，需要多个视角互相补充。

优势在于充分利用每个Agent的专长，思维碰撞容易产生创新想法，没有中心节点不会成为瓶颈。劣势是讨论可能陷入循环难以收敛，需要设计终止机制（如轮数限制、共识判断），且成本较高（每个Agent都需要处理大量上下文）。

流水线（Pipeline）

这种模式类似于工厂的生产线，任务按固定顺序流经各个Agent。

核心思想：将任务分解为多个阶段，每个Agent负责一个阶段，前一个Agent的输出是后一个Agent的输入。

工作流程：Agent 1接收原始输入，处理后传给Agent 2；Agent 2在Agent 1的基础上继续处理，传给Agent 3；依次类推，直到最后一个Agent输出最终结果。

适用场景：任务有明确的阶段划分，每个阶段的输入输出清晰，需要逐步精化的场景。

典型案例：文章创作流程——选题Agent → 大纲Agent → 初稿Agent → 润色Agent → 审核Agent。

优势在于逻辑清晰、易于理解和调试，每个阶段独立便于优化，可以针对每个环节做质量把控。劣势是缺乏灵活性，无法跳过或回退步骤；如果某个环节出错可能影响后续所有环节；前面环节的错误会被后续放大。

动态组合（Dynamic）

这是最灵活的模式，Agent的组合和顺序根据任务动态决定。

核心思想：有一个Router Agent根据任务类型和当前状态，动态选择调用哪个或哪些Agent，甚至可以多次调用、循环调用。

工作流程：Router Agent分析任务；根据任务特征选择合适的Agent；执行并评估结果；如果需要，调用其他Agent继续处理；重复直到任务完成。

适用场景：任务类型多样难以预设流程，需要根据中间结果调整策略，需要处理异常和回退。

典型案例：智能客服系统——Router根据用户问题类型，动态调用知识库Agent、订单查询Agent、人工转接Agent等，并根据回答效果决定是否需要补充调用其他Agent。

优势在于灵活性最高、适应复杂多变的场景，可以优化资源使用（只调用必要的Agent），支持迭代和自我修正。劣势是实现复杂度高，Router本身需要很强的决策能力，难以预测执行路径、调试困难。

下一节预告

理解了多Agent协作的各种模式，你可能已经意识到：无论采用哪种协作方式，Agent之间都需要高效的通信机制。下一节，我们将深入探讨Google提出的A2A协议——一个旨在标准化Agent间通信的框架，为你揭开多Agent系统底层通信的秘密。随后，我们将介绍专门为多Agent协作设计的CrewAI框架，让你能够快速构建复杂的多Agent系统。

图片

